Team Rayquaza: Transformer Fleet Health & Replacement Planning

HCLTech Future Ready AI Engineer Hackathon · Data Science & ML edition · 11 Oct 2026 · design brief

1 THE PROBLEM

> **In one line:** a power utility can only afford to fix a few transformers a year. We tell its engineer **which ones to replace or inspect first**, within the budget, and **why**.

The utility owns about 400 power transformers. If one fails, everyone it feeds loses power: sometimes a hospital, sometimes a village. Repairing after a failure is far more expensive than planned replacement. The data is **thin and uneven**: oil tests (gases dissolved in the transformer oil, the best health signal) exist for only about 60% of units, and old units may have just one test.

The user is the asset-planning engineer. **The decision** they make: given this year's budget, which transformers to replace, which to inspect, and which to leave alone. They need numbers they can trust and a reason for every choice.

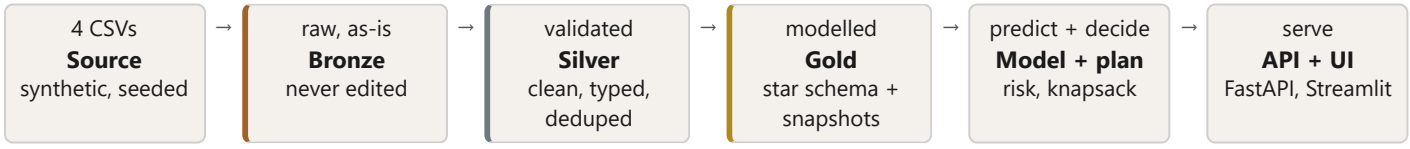
2 WHAT OUR SYSTEM DELIVERS

> **In one line:** for every transformer, a **trustworthy chance of failure**. From that, a **risk ranking** and a **budget plan**, each with its reasons.

THE ENGINEER ASKS	WHAT OUR SYSTEM ANSWERS
Failure probability	"TR-0102 has a **19%** chance of failing in the next 12 months (likely 12–28%)." A probability *and* a range, because certainty differs per unit.
Health score	A simple 0–100 score from the oil-gas readings, with the factors that drove it. Left blank, never "healthy", when there is no oil test.
Missing data	A unit with no oil test is still scored but flagged **low confidence** with a wide range, and is recommended for inspection.
Risk ranking	All transformers ranked by **risk = probability × impact**, with the reasoning for each.
Budget plan	"With 5 million, replace these 12 and inspect these 30." It fits the budget and removes more risk than replacing the oldest units.
Model quality	Honest test results on data the model never saw, compared with simple rules, including where it fails.

3 ARCHITECTURE: MEDALLION (RAW → CLEANED → BUSINESS-READY)

> **In one line:** data flows through three layers: **bronze** keeps it exactly as received, **silver** cleans and checks it, **gold** shapes it for analysis. Then the model, the plan and the API sit on top.



LAYER	WHAT HAPPENS	WHY IT EXISTS
Bronze	Each CSV is loaded **exactly as received** into its own table (`bronze_oil_tests`, `bronze_operation`, …), with two extra columns: when it arrived (`_ingested_at`) and which file it came from (`_source`). New files from the late data drop land here first.	The **replay tape**. If a later step has a bug, we fix it and re-run from bronze without re-fetching anything. It is also the audit trail of what arrived and when.
Silver	**Validate** first: HCL's checks run as hard `assert`s (no future dates, gas values positive, every transformer has a customer type, costs positive, row counts match the data card). Then **clean**: fix IDs (`"tr-0045 "` → `TR-0045`), remove duplicate assets and rows, correct gas **unit jumps** (a reading logged in ppb instead of ppm is 1000× too high), set proper types. Gaps stay **empty and flagged**, never silently filled.	The **one place** data quality is enforced. Everything after silver can trust the data. Bad data fails loudly here instead of quietly corrupting the model.
Gold	A **star schema**: `dim_transformer` (who it serves, customers, impact weight, region, size class) · `dim_date` (calendar) · `fact_oil_test` (one row per test) · `fact_operation` (one row per month of loading and temperature) · `fact_snapshot`, the table the model learns from (see the grain below).	Shaped for **reading**, not storing. Fewer joins, simpler queries. The snapshot table is the single, leakage-safe input for the model *and* the API.

Grain of the core table: one row of `fact_snapshot` = **one transformer at one quarterly snapshot date**. Picture standing on 1 Jan 2025 and asking: "*Using only what we knew that day, will this transformer fail in the next 12 months?*" Every input uses data from **before** the date. The answer (the label) comes from the 12 months **after** it. This is how we stop the model from "seeing the future", the mistake HCL disqualifies teams for. The "latest test before the date" and "change since the previous test" are built with SQL window functions (`ROW_NUMBER`, `LAG`).

4 THE DATA: WE GENERATE ALL FOUR FILES

> **In one line:** HCL provides no data, so we generate a **realistic, reproducible** fleet with every trap from the guide planted on purpose, so we can show each one being handled.

What we generate: 400 transformers over 36 months (Oct 2023 – Sep 2026). Monthly loading, oil temperature and age; oil-gas tests for about 62% of units (38% never tested); who each unit serves; and action costs plus the annual budget. Gas readings follow textbook fault patterns: overheating raises ethylene and methane, electrical arcing raises acetylene. Failures follow a **written-down rule**: older, overloaded units with a developing gas fault fail more often, plus randomness. Fixed random seed, so the same data comes out every run. A data card documents everything.

EDGE CASE IN THE GUIDE (PLANTED)	HOW OUR SYSTEM HANDLES IT
Transformers with no oil test at all	A "has oil test" flag feeds the model; result marked **low confidence** with a wide range; never called healthy; inspected before any replacement.
Old transformers with few measurements	Wider uncertainty range and a low-confidence status; the plan prefers inspecting them first.
Look-alike pair: hospital vs rural	Same probability, very different **impact**, so the hospital unit ranks far higher in risk.
Gas values jump because of a unit change	Silver detects a ~1000× jump against that unit's history, rescales it and flags it.
Recent repair resets the pattern	Only oil tests taken **after** the last repair count towards current health.
Budget cannot cover all high-risk units	The optimiser picks the best risk reduction per unit of money; the rest are listed as deferred.

5 THE MODEL: WHAT WE TRAIN AND WHAT IT IS FOR

> **In one line:** we train a model that reads a transformer's history and outputs its **chance of failing in the next 12 months**. We keep it simple, test it honestly, and make sure its probabilities can be trusted.

What it learns from

- **Each training example** = one transformer at one snapshot date, with the answer: did it fail in the next 12 months?
- **Inputs** in plain terms:
  - **Oil-gas readings**: latest levels, gas ratios that indicate the fault type, and how fast gases are rising
  - **Stress**: average and peak loading, months running hot
  - **Age**, and how stale our information is (months since the last test)
  - **Whether it has ever been tested**: being untested may itself signal risk

What we train

- **Two simple baselines** to beat: an **age-only** rule ("older = riskier") and a hand-written **health index** from the gas readings.
- **Logistic regression**: each input gets a weight, and the weighted sum becomes a probability. Fully readable. Our expected winner, because there are few failures to learn from.
- **Gradient boosting** (many small decision trees) for comparison. It is kept only if it clearly beats the simpler model.
- We compare **three ways of handling missing readings**, as the guide asks, and pick the best by the numbers.

How we make it trustworthy

- **Honest testing:** train on earlier dates, test on later dates, and on transformers the model has never seen, the way it will be used in real life. The final test period is locked away and used once.
- **Calibration:** when the model says 20%, about 20 in 100 such units should really fail. This matters because the risk formula multiplies the probability directly.
- **Uncertainty range:** each unit gets a range, not just a number. It is wider when we know less (no test, old test).
- **Explanations:** for every transformer, the top factors behind its score (e.g. "acetylene rising + peak load 95%").

How we measure it

- Are the real failures near the **top** of our list? (PR-AUC, and top-10% recall)
- Are the **probabilities honest**? (Brier score, reliability chart)
- Results for **never-tested units reported separately**.
- Accuracy is **not** used: "nothing will fail" is about 96% accurate and useless.

6 FROM PREDICTION TO DECISION: RISK AND THE BUDGET PLAN

> **In one line:** risk = **chance of failure × who loses power**. An optimiser then picks the set of replacements and inspections that removes the most risk without exceeding the budget.

Worked example (HCL's rule: hospital 5, industry 3, urban 2, rural 1, × customers): TR-0102 serves a hospital with 12,000 customers: impact 60,000, failure chance 19% → **risk 11,400**. TR-0117 serves a rural area with 2,000 customers: impact 2,000, failure chance 25% → **risk 500**. TR-0102 comes first even though TR-0117 is more likely to fail.

- **Replace** when risk is high *and* we know the unit well (recent oil tests). A new unit removes almost all of its risk.
- **Inspect** when we know little (no test, old test, wide range). Inspection is cheap, and its value is the hidden risk it could reveal. **We never spend on a blind replacement:** an untested unit is inspected first.

- **Choosing within the budget** is a classic **knapsack** problem: each action has a cost and a value (risk removed), and we pick the best combination that fits. Each transformer gets at most one action.
- We compare against an **age-based plan** (replace the oldest until the money runs out) and show how much more risk our plan removes.
- **Every plan is a proposal.** The engineer approves it.

Mentor feedback we acted on (12:00): "*If an inspection shows the transformer is worse than predicted, its risk should go up.*" Our design: an inspection **is** a new oil test. Its result is loaded (`/ingest`), the pipeline re-runs, the model re-scores, and if the unit is worse its probability rises and its range narrows. The next plan then **replaces it automatically** if the budget allows. If it is fine, it drops off the list. The 13:45 late data drop (new oil tests) shows exactly this loop live.

7 HOW IT RUNS AND HOW JUDGES USE IT

One command, safe to repeat

- `python -m src.pipeline` runs load → bronze → validate + clean → silver → gold, printing one status line per stage.
- **Re-runnable:** silver and gold are rebuilt in full, so running twice gives identical results.
- **All-or-nothing writes:** gold is written in one transaction; a failure rolls back.
- `docker compose up` starts the API and the UI.

ENDPOINT	WHAT IT DOES
`POST /predict`	scores transformers: probability, range, drivers
`POST /plan`	budget plan vs the age-based plan
`POST /ingest`	loads a new CSV, then re-runs
`GET /model/info`	model card: method, data window, metrics
`GET /explain/{id}`	why this unit scored what it did
`GET /health`	API, database and model status

8 KEY DESIGN DECISIONS: WHAT WE CHOSE AND WHAT WE REJECTED

DECISION	REJECTED	WHY
Medallion layers	one script, raw → model	Raw data is kept, so any step can be replayed and audited; quality is enforced in one place.
Star schema in gold	fully normalised (3NF)	Gold is read constantly and written once, so fewer joins matter more than avoiding some repetition.
Quarterly snapshots	monthly / yearly	Oil tests come every 6–12 months, so monthly rows would be near-duplicates and yearly gives too few examples.
Simple model first	complex model by default	With few failures, complex models memorise noise. A simple model must be beaten before anything replaces it.
Full refresh	incremental loads	About 15k rows rebuild in seconds; incremental would need a watermark and adds failure modes.
SQLite	PostgreSQL server	No server to fail on venue wifi; window functions cover our queries; fine at this size.
Inspect before replacing	replace on prediction alone	A replacement is ~25× the cost of an inspection; never spend it on a unit we have not looked at.

9 WHO BUILDS WHAT

MEMBER	OWNS
A: Data & Serving	synthetic data generator and data card; FastAPI endpoints; Streamlit UI; Docker
B: Architecture (Unique)	bronze → silver → gold, validation, the shared feature function (used by training *and* the API), loader, README
C: Model & Plan	baselines, models, honest testing, calibration, explanations; risk and the knapsack plan; evaluation report

All three build against one written contract (every table, column and function agreed up front), so work runs in parallel and joins cleanly.